

40 Chunking Interview Questions

The RAG component engineers underestimate the most.
Your retrieval quality depends on: **how you chunk.**

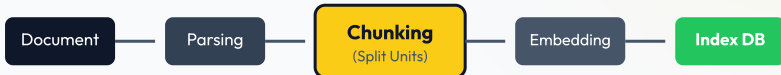
SIZING & STRATEGIES COVERED:

Fixed-size — Recursive — Structure-aware — Semantic — Parent-child — Multimodal —
Evaluative

40 Q&As — Sizing Rules — Failures →

What is chunking?

Chunking is the process of splitting a larger document into smaller units that can be independently processed, embedded, indexed, and retrieved.

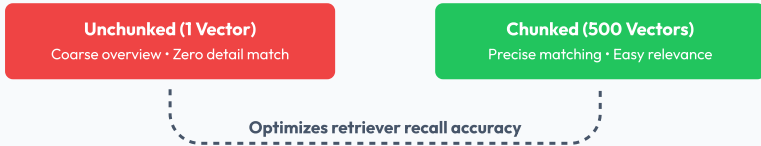


ENGINEERING AXIOM

Chunking is an information-retrieval design decision, not just a simple text preprocessing step.

Why is chunking **important**?

Embedding a 100-page document as a single vector prevents granular searches. Split it into topic-focused blocks:



OBJECTIVE

Ensure query metrics find exact policy passages instead of averaging entire document coordinates.

What makes a **good chunk**?

A good chunk represents a coherent, self-contained unit of information.

- ✖ **Poor Chunk:** Section heading + half a paragraph + truncated table header.
- ✔ **Better Chunk:** Unified section headers, policy conditions, and exception details kept together.

RULE OF THUMB

Target semantic completeness over arbitrary character size thresholds. Chunks must convey continuous meaning.

If chunks are **too small**?

Splitting text too aggressively isolates sentences, resulting in fragmented information context:



Fragments lose connection — forces model to guess details

CONS

High search database indexing overhead, broken semantic relationships, and increased hallucination risks.

If chunks are **too large**?

Overly massive document chunks merge separate topics together, degrading search accuracy:

Large 15-page Segment Chunk

(Travel rules • Payroll limits • Security rules mixed)

Retrieves excessive noise — inflates token bills

CONS

Low search precision, high latency, prompt attention dilution, and increased API costs.

How to choose chunk size?

Do not copy generic tutorial settings. Evaluate sizing dimensions systemically:



STANDARD PRACTICE

Run evaluations on your golden dataset comparing different chunk size configurations (e.g. 256 vs 512 tokens).

Should size be identical?

No. Natural text structures vary across document categories. Apply custom chunking rules accordingly:

- 🔍 **FAQs:** Ground matching Q&A pairs together.
- 📌 **Contracts:** Retain sections as cohesive clauses.
- </> **Code:** Maintain class and function boundaries.
- 🗣️ **Transcripts:** Split at topic/speaker shift points.

ARCHITECTURAL RULE

Deploy document-type-aware routing in your ingestion pipeline to parse different files appropriately.

Token-based vs character-based

Comparing splitting strategies across data types:

- **Character**-Splits text strictly by character count boundaries. Easy to **based:** implement, but character density doesn't directly represent LLM token usage.
- **Token**-Splits text based on model vocabulary tokens. Aligns perfectly with **based:** context budgets and costs, though tokenizers vary across models.

TAKEAWAY

If your application has strict latency or token budget constraints, use token-aware splitting to prevent overflow errors.

What is **chunk overlap**?

Overlap copies trailing segments from previous blocks into adjacent ones to maintain semantic continuity:



PURPOSE

Prevent important statements from being cut in half right at the chunk boundary.

Why is overlap **useful** ?

Consider a key policy sentence split across a chunk boundary:

- **Without** Chunk "Eligible . "12 months . Neither contains the **Overlap**: I has after Chunk of complete meaning completing" 2 has service." independently.
- **With** Both chunks retain the boundary context, allowing search models **Overlap**: to match query semantics properly.

CONTINUITY

Overlap ensures semantic search models can match query syntax even if target keywords sit right on boundaries.

Can too much overlap hurt?

Excessive overlap replicates duplicate information across vectors, degrading search quality:



CONS

Increases database storage costs, wastes prompt tokens, and reduces search result diversity.

How to choose overlap?

Start with document structure rules, then run experiments to tune it:

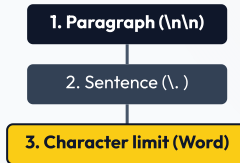
- > **0% Overlap:** Test baseline. Low storage, but boundary data gets lost.
- > **10%–20% Overlap:** Balanced sweet spot. Preserves context boundary phrases.
- > **>30% Overlap:** High replication. Increases prompt costs.

EXPERIMENT RULE

Run query benchmarks using different overlap percentages on your golden dataset to optimize cost vs accuracy ratios.

What is **recursive chunking**?

Recursive chunking splits text using progressively smaller separators until size constraints are satisfied:



ADVANTAGE

It avoids splitting sentences or paragraphs abruptly, keeping logical ideas grouped together.

Why use **recursive chunking**?

It balances simple character-based splits with fully semantic chunking pipelines:

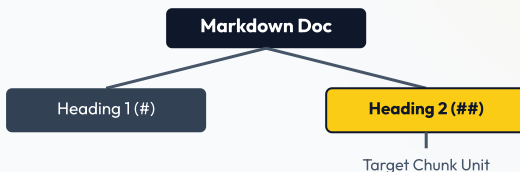
- ✓ Simple to implement, using standard libraries (like LangChain's RecursiveCharacterTextSplitter).
- ✓ Highly predictable results, avoiding API-dependency risks.
- ✓ Serves as an excellent baseline configuration for most textual datasets.

INGESTION BASELINE

Always establish your RAG baseline using recursive chunking before attempting expensive semantic parsing.

What is **structure chunking**?

Structure-aware chunking parses documents using their inherent formatting boundaries:



GOAL

Preserve logical boundaries (like titles and tags) that authors originally designed to group ideas.

Why does **structure** win?

Comparing splitting strategies on layout documents:

- **Fixed-Size splits:** Cuts text blindly, mixing introduction sections with policy text.
- **Structure-Aware splits:** Respects headings (e.g. # Leave Policy), keeping the entire section together.

IMPORTANT LIMITATION

Structure-aware chunking must still enforce size caps — a single heading section can be too large for efficient context windows.



What is semantic chunking?

Semantic chunking groups sentences based on their logical meaning rather than token counts:



TECHNIQUE

Calculate cosine similarity scores between consecutive sentences; trigger a chunk split when scores drop below a threshold.

Advantages of semantic splits

Semantic chunking preserves conceptual coherence by separating topics dynamically:

- + Prevents single chunks from mixing unrelated concepts.
- Requires running sentence embeddings, increasing parsing time and cost.
- ! Performance relies heavily on the quality of the embedding model.

FACT

Use semantic chunking for long, unstructured documents (like prose or essay streams) where formatting tags are missing.

When is it **justified**?

Evaluate semantic chunking based on document structure:

- | | | |
|-------------------------------|---|---|
| ✓ Unstructured Text: | High value. Paragraphs shift topics without headings or tags. | |
| ✗ Structured Markdown: | Low value. Heading boundaries (e.g. | #, ##) already provide clean semantic splits. |

INTERVIEW TAKEAWAY

Always choose the simplest chunking strategy that delivers good search results.
Avoid over-engineering.

Can semantic splits solve everything?

No. Ingestion pipelines fail at multiple layers. Good chunking cannot compensate for errors downstream:

- **Upstream:** Broken PDF parsers extract corrupted text.
- **Downstream:** Weak query formulations fail vector database matches.

SYSTEM PRINCIPLE

Evaluate chunking strategy changes within the context of your entire RAG pipeline's retrieval accuracy.

Chunking legal documents

Legal documents rely on strict hierarchical clause numbering networks:

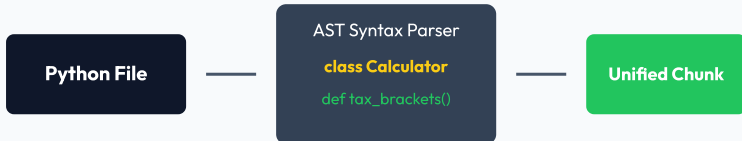
- ✓ Extract clause tags (e.g. Section 8.2(a)) as metadata attributes.
- ✓ Keep section titles grouped directly with clauses.
- ✓ Retain definition clauses to resolve ambiguous terms.

BENEFIT

Enables exact clause citations, helping downstream LLMs refer back to specific pages or sections accurately.

Chunking **source code**

Do not split code like ordinary text. Maintain code block syntax boundaries:

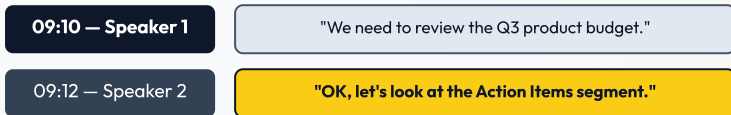


RULE

Use Abstract Syntax Tree (AST) parsers to chunk at class or method boundaries, appending file metadata.

Chunking meeting transcripts

Transcripts are sequential conversation flows. Standard character splits can cut speakers off mid-sentence:



Group by Speaker Turn & Topic boundaries

METADATA

Always append speaker names and timestamps to transcript chunks to preserve conversational context.

Chunking research papers

Academic research papers follow a standardized section hierarchy. Leverage this in your parser:

- Segment chunks by sections (e.g. Abstract, Methodology, Results).
- Keep inline citations intact to preserve references.
- Extract formula blocks (using LaTeX) as self-contained chunks.

WHY?

Enables answering queries like "What dataset was used?" without pulling in the entire paper.



Why are **tables** difficult?

Standard text splitters flatten table rows and columns, turning relationships into ambiguous text string runs:

Flat Parse (Bad)

"Region Revenue India 10M"

JSON Parse (Good)

```
{"Region": "India", "Rev": "10M"}
```

Flat parsing destroys cross-column relationships

SOLUTION

Use table-aware parsers to convert tabular grids into structured JSON arrays or Markdown layouts before chunking.



Should a table be **one chunk**?

Not necessarily. Table chunking strategy depends on cell sizes:

- ✓ **Small** Keep the entire table unified within a single chunk to preserve
Tables: cell coordinates.
- ✓ **Huge** Split by logical row chunks, and prepend column header titles to
Tables: every row segment.

TIP

If users regularly query row-level data, configure row-based indexing instead of embedding the entire table.



Handling images in RAG

Images contain valuable information that text-based extractors miss.
Implement a multimodal pipeline:



METHODOLOGY

Run charts and figures through vision models to generate text captions, OCR scripts, or visual embeddings before indexing.

Impact of **poor PDF parsing**

A bad PDF parser corrupts formatting layouts, destroying the quality of downstream components:



TAKEAWAY

Always verify document extraction outputs before blaming vector database retrieval algorithms for bad responses.

What **metadata** to store?

Enrich chunks with structured metadata to enable filtering and tracing:

- **Source details:**document_id, title, page, source_url.
- **Hierarchy:**section headings, parent_id.
- **Security keys:**tenant_id, access_level permissions.

WHY?

Enables metadata filtering, access control policies, and accurate citation mapping in answers.

Should metadata be **embedded**?

Embedding all metadata as raw text can dilute similarity coordinates and introduce noise.

- ✓ **Use Embeddings for:** Semantic prose content (queries look for meaning match).
- ✓ **Use Filters for:** Structured metadata flags (e.g. department = HR).

DESIGN RULE

Keep metadata values in database columns for strict filtering. Only embed text that helps search similarity matches.

What is contextual chunking?

Prepend parent document summaries to standalone chunks to preserve semantic context:

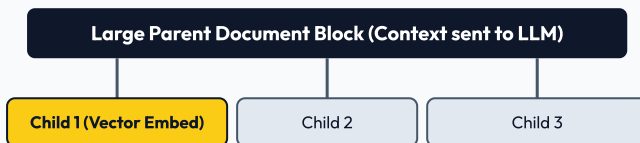


BENEFIT

Drastically improves vector database matching for isolated sentences that lack explicit keywords.

Why use **parent-child** links?

Isolate retrieval from generation sizing constraints. Search small child chunks, but return the parent context to the model:



RESULT

High search match accuracy combined with complete context for the downstream reasoning model.

Retriever misses facts?

If search queries fail to retrieve correct information, evaluate if chunk boundaries are responsible:

- ✖ Information needed to answer queries is split across separate chunks.
- ✖ Section headers were separated from the main policy paragraphs.
- ✖ Overlap sizes were too small to preserve context boundaries.

DIAGNOSIS

Verify if the expected information is present in the database index, then trace which chunk contains it.

How to **evaluate** chunking?

Benchmark chunk configurations systematically on your evaluation dataset:



STANDARD

Choose the setup that yields the highest recall and precision scores within your cost budget limits.

Is it chunking or embeddings?

Isolate your pipeline variables sequentially to locate errors:

- ✓ **Test** Keep embedding models constant. Vary chunking strategies, and
 - 1: evaluate database search recall.
- ✓ **Test** Keep chunking strategies constant. Change embedding models, and
 - 2: compare search results.

INTERVIEW INSIGHT

Do not change both at the same time. Changing multiple variables prevents you from identifying what improved the system.

Signs chunks are **too large**

Common signs that chunk size limits are set too high:

- ❗ Retrieved chunks contain multiple unrelated topics.
- ❗ Input context token costs are high.
- ❗ Downstream models overlook key facts or get confused.
- ❗ Output answers contain irrelevant details.

DIAGNOSTIC RULE

Calculate: "What percentage of the retrieved text actually helped answer the query?" If low, reduce chunk size.

Signs chunks are **too small**

Common signs that chunk size limits are set too low:

- ⚠️ Answers require combining details from many scattered chunks.
- ⚠️ Explanations get separated from their core definitions.
- ⚠️ Top-K parameters must be set very high to return complete context.

RESULT

Setting chunks too small forces you to retrieve more blocks, increasing token costs and complexity.

Can chunking cause hallucinations?

Yes. If exception clauses are split from eligibility statements (e.g. "12 months service required" in Chunk 1, and "Exceptions for contractors" in Chunk 2), search retrieval may only return the first block.

The downstream model, lacking the exceptions context, will generate an incomplete and factually incorrect answer.

TAKEAWAY

Chunking directly affects grounding performance — complete context is needed to prevent models from guessing.

Static vs dynamic chunking

Comparing sizing strategies during ingestion vs query time:

- **Static:** splits text files during ingestion. Simple and fast, but lacks flexibility.
- **Dynamic:** shifts context boundaries dynamically based on the query. Highly flexible, but increases compute costs and latency.

STANDARD PRACTICE

Deploy static ingestion indexing first. Use parent-child retrieval rules at query time to adjust context dynamically.

Fix chunk size **first**?

Not necessarily. Diagnose the root cause of failures before changing parameters:

- Is the target information present in the source files?
- Was text parsed cleanly by the document reader?
- Are embedding vectors capturing query concepts?

STRONG ANSWER

"I would first diagnose if the failure is in parsing, retrieval, or generation. Then, I would tune one parameter at a time."

How should I chunk this?

Follow this routing logic based on your document structure:

Data: Code

AST Node → functions/classes chunking.

Data: Tables

Row-aware / structured JSON schemas.

Structured Prose

Structure-aware heading tags (H1/H2) splits.

Unstructured Prose

Recursive split baseline or semantic splits.

Is chunk too large?

Yes: Split further

No: Add context info

RULE OF THUMB

If retrieval quality is poor, increase overlap, append parent context summaries, or test parent-child retrieval.

Don't guess. Evaluate.

Configure systematic experiments across three core variables:

1. Chunk Size

300 / 500 / 800 / 1200
tokens.

2. Overlap

0% / 10% / 20% token
density.

3. Separators

Fixed / Recursive /
Semantic.



KEY PRINCIPLE

Change only one variable at a time so you know what improved search accuracy.

No "Best Chunk Size" exists!

Be ready to answer the engineering decisions:

- **Define** an evaluation baseline dataset.
- **Test** multiple chunking sizing strategies.
- **Measure** retrieval recall and answer precision.
- **Choose** the configuration optimizing cost-latency limits.

"Engineering a RAG system means benchmarking quality-cost trade-offs."

✓ Day 3 Complete

🔖 Save this series

📖 **Next:** 40 AI & LLM Security Interview Questions